

Dokumentacja

Dashboard do wizualizacji danych SQL z Archiwum



SOLARIS
NARODOWE CENTRUM
PROMIENIOWANIA
SYNCHROTRONOWEGO

Miejsce	Narodowe Centrum Promieniowania Synchrotronowego SOLARIS
Autor:	Karol Ożóg
Data:	Wrzesień 2026
Do dokumentacji:	https://www.overleaf.com/4254967931ntwnqptwqdcv#77ef8f
Kod:	https://github.com/karolozog15/dashboard_solaris.git

Aplikacja webowa umożliwiająca przeglądanie i wizualizację archiwalnych danych pomiarowych przechowywanych w bazach Microsoft SQL Server. Została wykonana w języku Python z wykorzystaniem frameworka Streamlit oraz bibliotek `pandas`, `pyodbc` i `plotly`. Projekt zawiera również skrypt `start.vbs`, który ułatwia uruchamianie aplikacji w systemie Windows.

Kraków, 2026

Spis treści

1	Przygotowanie i uruchomienie aplikacji	1
1.1	Wymagania	1
1.2	Pliki projektu	1
1.3	Instalacja i uruchomienie z terminala	1
1.4	Instalacja bez dostępu do Internetu	1
1.5	Uruchamianie przez <code>start.vbs</code>	2
2	Opis działania aplikacji	2
2.1	Główny przepływ aplikacji	2
2.2	Wybór bazy danych	2
2.3	Wybór tabel i zmiennych	2
2.4	Konfiguracja serii	2
2.5	Zakres czasu	3
2.6	Odświeżanie i prezentacja danych	3
3	Moduł <code>config.py</code>	3
3.1	Połączenie, czas i limity	3
3.2	Kolory serii	3
3.3	Ikony	3
4	Moduł <code>db.py</code>	3
4.1	Pooling i tworzenie połączeń	4
4.2	Nazwy schematów i tabel	4
4.3	Pobieranie baz, tabel i zmiennych	4
4.4	Nazwy opisowe zmiennych	4
4.5	Pobieranie i agregacja danych	4
4.5.1	Filtrowanie i obliczenie czasu	4
4.5.2	Wyznaczanie granic i bucketów	5
4.5.3	Agregowanie wartości	5
4.6	Konwersja wyniku w Pandas	5
4.7	Cache zapytań	5
5	Moduł <code>state.py</code>	5
5.1	Inicjalizacja stanu sesji	5
5.2	Tryby wyboru zakresu	6
5.3	Zapamiętywanie zakresu czasu	6
5.4	Dodawanie, usuwanie i resetowanie serii	6
5.5	Zaznaczenie zakresu i historia zoomu	6
5.6	Cofanie zoomu	6
6	Moduł <code>charting.py</code>	6
6.1	Klucze wykresu i stan widgetów	7
6.2	Konwersja i formatowanie czasu	7
6.3	Zakresy i rozmieszczenie osi Y	7
6.4	Budowa figury Plotly	7
6.5	Ślady AVG, MIN i MAX	7
6.6	Podsumowanie serii	7
6.7	Odczyt zdarzeń wykresu	8
6.8	Markery M1 i M2	8
6.9	Zaznaczanie zakresu i zoom	8
6.10	Sterowanie myszy i klawisza spacji w <code>controls.html</code>	8
6.11	Fragment Streamlit i odświeżanie	8

1 Przygotowanie i uruchomienie aplikacji

Aplikacja służy do przeglądania i wizualizacji historycznych danych pomiarowych zapisanych w Microsoft SQL Server. Interfejs jest zbudowany w Streamlit, a wykresy tworzone są w Plotly.

1.1 Wymagania

Do uruchomienia aplikacji potrzebne są:

- Python w wersji 3.10 lub nowszej,
- dostęp do serwera Microsoft SQL Server i uprawnienia do odczytu,
- sterownik ODBC Driver 17 for SQL Server,
- Windows i uwierzytelnianie Windows dla konfiguracji połączenia używanej w projekcie,
- biblioteki Python: **streamlit**, **pandas**, **plotly** i **pyodbc**.

Skrypt **start.vbs** wymaga systemu Windows. Sama aplikacja Streamlit nie korzysta z VBScript, ale aktualna konfiguracja połączenia z bazą używa uwierzytelniania Windows.

1.2 Pliki projektu

Główne pliki projektu to:

- **app.py** – interfejs i główny przepływ aplikacji,
- **db.py** – połączenie z SQL Serverem, zapytania i agregacja danych,
- **charting.py** – wykresy, osie, markery i podsumowania,
- **state.py** – stan sesji, serie i zakresy czasu,
- **config.py** – ustawienia połączenia, limity i kolory,
- **controls.html** – dodatkowe sterowanie wykresem za pomocą myszy i klawisza spacji,
- **start.vbs** – skrypt uruchomieniowy dla Windows,
- **README.md** – skrócona dokumentacja projektu.

1.3 Instalacja i uruchomienie z terminala

W katalogu projektu należy utworzyć środowisko wirtualne:

```
python -m venv .venv
```

Następnie można zainstalować zależności:

```
.\.venv\Scripts\python.exe -m pip install streamlit pandas plotly pyodbc
```

Aplikację uruchamia się poleceniem:

```
.\.venv\Scripts\python.exe -m streamlit run .\app.py
```

Streamlit domyślnie uruchamia aplikację pod adresem:

```
http://localhost:8501
```

Adres serwera SQL Server i nazwa domyślnej bazy są określone w **config.py**. Szczegóły tworzenia połączenia i zapytania SQL znajdują się w **db.py**.

1.4 Instalacja bez dostępu do Internetu

Na komputerze z dostępem do Internetu można pobrać pakiety i ich zależności do katalogu **packages**:

```
py -m pip download -d .\packages streamlit pandas plotly pyodbc
```

Katalog **packages** należy przenieść na komputer docelowy. Pakiety można tam zainstalować bez dostępu do Internetu:

```
.\.venv\Scripts\python.exe -m pip install --no-index --find-links=.\packages streamlit pandas plotly pyodbc
```

Pakiety pobrane do instalacji offline muszą być zgodne z wersją Pythona i architekturą używaną na komputerze docelowym.

1.5 Uruchamianie przez `start.vbs`

W systemie Windows aplikację można uruchomić przez plik `start.vbs`. Skrypt oczekuje interpretera:

```
.venv\Scripts\python.exe
```

oraz pliku `app.py`. Sprawdza, czy port 8501 jest zajęty. Jeśli port jest wolny, uruchamia Streamlit i otwiera lokalny adres aplikacji w przeglądarce.

Skrypt zapisuje adres lokalny i wykryty adres sieciowy do pliku `link.txt`. Dostępność adresu sieciowego zależy m.in. od konfiguracji komputera i zapory sieciowej.

Jeżeli port 8501 jest zajęty, skrypt sprawdza nazwę procesu, ścieżkę interpretera i linię polecenia. Proces jest kończony tylko wtedy, gdy zostanie rozpoznany jako oczekiwana instancja aplikacji. W przeciwnym przypadku skrypt wyświetla ostrzeżenie i nie zabija procesu. Po udanym zakończeniu starej instancji uruchamiana jest nowa.

2 Opis działania aplikacji

2.1 Główny przepływ aplikacji

Plik `app.py` jest głównym punktem wejścia aplikacji. Odpowiada za konfigurację strony, wyświetlenie interfejsu, pobranie ustawień użytkownika, pobranie danych oraz przekazanie ich do modułu wykresów.

Główny przebieg działania obejmuje:

1. konfigurację strony Streamlit i inicjalizację stanu sesji,
2. pobranie listy baz i wybór bazy,
3. pobranie tabel dostępnych w wybranej bazie,
4. pobranie nazw opisowych zmiennych, jeśli są dostępne,
5. wybór i konfigurację serii danych,
6. wybór zakresu czasu i częstotliwości odświeżania,
7. pobranie oraz zagregowanie danych,
8. wyświetlenie wykresu, podsumowania i opcjonalnej tabeli danych.

Strona Streamlit jest wyświetlana w szerokim układzie. Po inicjalizacji stanu sesji aplikacja pokazuje tytuł i opis dashboardu.

2.2 Wybór bazy danych

Listę dostępnych baz pobiera funkcja `load_databases()` z modułu `db.py`. Domyślnie wybierana jest baza określona przez `DEFAULT_DB_NAME`, obecnie `Archives`. Jeśli nie ma jej na liście, aplikacja wybiera pierwszą dostępną bazę.

Jeżeli nie uda się pobrać listy baz albo lista jest pusta, aplikacja wyświetla odpowiedni komunikat i zatrzymuje dalsze działanie.

Po zmianie bazy wywoływana jest funkcja `reset_series()`. Usuwa ona konfigurację serii, ponieważ wybrane wcześniej tabele i zmienne mogą nie istnieć w nowej bazie.

2.3 Wybór tabel i zmiennych

Po wybraniu bazy aplikacja pobiera listę tabel. Są one prezentowane w formacie `SCHEMA.TABLE`, na przykład `dbo.POMIARY`. Po wybraniu tabeli pobierane są dostępne wartości kolumny `VARIABLE`.

Nazwy opisowe zmiennych są opcjonalnie pobierane z tabeli określonej przez `NAZWY_TABLE` w `config.py`. Jeśli pobranie mapowania nie powiedzie się, aplikacja nadal działa i pokazuje same identyfikatory zmiennych.

2.4 Konfiguracja serii

Użytkownik może wybrać maksymalnie cztery serie. Dla każdej serii konfiguruje:

- tabelę i zmienną,
- automatyczny lub ręczny zakres osi Y,
- dolną i górną granicę osi Y w trybie ręcznym.

Serie można dodawać i usuwać w panelu bocznym. Każda seria ma osobny identyfikator UUID, dzięki czemu ustawienia widgetów jednej serii nie kolidują z ustawieniami pozostałych.

2.5 Zakres czasu

Aplikacja udostępnia trzy tryby określania zakresu:

- **Ostatnie dyskretne** – wybór gotowego przedziału,
- **Ostatnie ciągłe** – podanie liczby sekund, minut, godzin i dni wstecz,
- **Własny zakres** – wybór daty i godziny początku oraz końca.

Gotowe zakresy to 10 minut, 1 godzina, 1 dzień, 1 tydzień, 1 miesiąc (30 dni) i 1 rok (365 dni). Domyślnym gotowym zakresem jest 1 godzina.

Jeżeli początek zakresu jest późniejszy od końca, aplikacja wyświetla błąd i nie pobiera danych. Zaznaczenie fragmentu wykresu może zmienić aktualny zakres; przycisk cofania przywraca poprzedni poziom zoomu albo zakres początkowy.

2.6 Odświeżanie i prezentacja danych

Dane można odświeżać ręcznie albo okresowo: co 1, 5, 15 lub 30 minut. Przycisk „Odśwież teraz” czyści cache danych pomiarowych i ponownie uruchamia aplikację.

Wykres pokazuje wartości AVG, a opcjonalnie także MIN i MAX. Każda seria ma własną oś Y. Pod wykresem znajduje się tabela podsumowująca serie. Osobno można włączyć tabelę danych zagregowanych; zawiera ona maksymalnie 1000 najnowszych rekordów na serię.

3 Moduł config.py

Plik `config.py` zawiera najważniejsze stałe konfiguracyjne: ustawienia połączenia z bazą, domyślną bazę, limity aplikacji, czas przechowywania danych w cache oraz kolory serii.

3.1 Połączenie, czas i limity

Konfiguracja bazy obejmuje:

```
DB_SERVER = r"ZENON14WIN\ZENON_2022"
DEFAULT_DB_NAME = "Archives"
DB_DRIVER = "ODBC Driver 17 for SQL Server"
NAZWY_TABLE = "dbo.WODA_VARIABLES"
```

Strefa czasu jest zdefiniowana jako stałe przesunięcie UTC+1. Nie jest to strefa automatycznie uwzględniająca zmianę czasu letniego i zimowego.

Najważniejsze limity oraz czasy cache to:

```
MAX_PLOT_POINTS = 1_000
MAX_TABLE_ROWS = 1_000
MAX_SERIES = 4

DATA_CACHE_TTL = 300
VARIABLE_CACHE_TTL = 600
TABLE_CACHE_TTL = 600
```

Dane pomiarowe są przechowywane w cache przez 300 sekund, a listy baz, tabel i zmiennych przez 600 sekund. Szczegółowe limity wpisów cache są ustawione bezpośrednio w dekoratorach funkcji w `db.py`.

Stałe `AXIS_GAP` i `MAX_DOMAIN_START` wpływają na rozmieszczenie dodatkowych osi Y i szerokość obszaru wykresu.

3.2 Kolory serii

Lista `VARIABLE_COLORS` zawiera kolory wykorzystywane do odróżniania serii. Funkcja `get_series_color(index)` wybiera kolor na podstawie indeksu serii. Jeśli indeks przekroczy liczbę kolorów, paleta jest używana cyklicznie.

3.3 Ikony

W `config.py` zapisane są ścieżki do ikon wykorzystywanych przez streamlit w celu ich użycia w innych miejscach

4 Moduł db.py

Moduł `db.py` odpowiada za połączenie z SQL Serverem, pobieranie metadanych bazy, zapytania o dane pomiarowe oraz ich agregację. Wyniki funkcji pobierających dane są przechowywane w cache Streamlit.

4.1 Pooling i tworzenie połączeń

Moduł włącza pooling połączeń biblioteki pyodbc:

```
pyodbc.pooling = True
```

Połączenie tworzy funkcja `get_connection(database)`. Korzysta ona z ustawień serwera i sterownika z `config.py`. Jeśli nazwa bazy nie zostanie podana, używana jest baza `master`.

Połączenie wykorzystuje uwierzytelnianie Windows (`Trusted_Connection=yes`), nie wymusza szyfrowania (`Encrypt=no`) i ma 10-sekundowy limit na nawiązanie połączenia. Funkcje wykonujące zapytania zamykają połączenie w bloku `finally`.

4.2 Nazwy schematów i tabel

Funkcja `quote_identifier(value)` przygotowuje nazwę schematu lub tabeli do umieszczenia w zapytaniu SQL. Usuwa skrajne spacje, odrzuca pustą nazwę oraz nazwy zawierające znak `]`. Poprawną nazwę obejmuje nawiasami kwadratowymi, na przykład `dbo` zostaje zapisane jako `[dbo]`.

Nazwy obiektów SQL nie są przekazywane jako parametry zapytania, dlatego są walidowane przed wstawieniem do zapytania. Wartości takie jak identyfikator zmiennej i granice czasu są przekazywane osobno jako parametry.

4.3 Pobieranie baz, tabel i zmiennych

Funkcja `load_databases()` pobiera bazy o stanie `ONLINE` z tabeli systemowej `sys.databases`. Warunek `database_id > 4` pomija standardowe bazy systemowe `master`, `tempdb`, `model` i `msdb`. Funkcja zwraca listę nazw baz.

Funkcja `load_tables(database)` pobiera z `INFORMATION_SCHEMA.TABLES` tabele bazowe, pomijając widoki. Zwraca `DataFrame` z kolumnami `TABLE_SCHEMA` i `TABLE_NAME`.

Funkcja `load_variables(database, table_schema, table_name)` pobiera unikalne, niepuste wartości kolumny `VARIABLE` z wybranej tabeli i zwraca je w kolejności alfabetycznej.

4.4 Nazwy opisowe zmiennych

Funkcja `load_variable_names(database)` pobiera kolumny `VARIABLE` i `NAME` z tabeli wskazanej przez `NAZWY_TABLE`. Wynik jest przekształcany na słownik, w którym identyfikator zmiennej jest kluczem, a jej nazwa opisowa wartością.

Funkcja `get_variable_label(variable, names_map)` zwraca etykietę w postaci `Nazwa (IDENTYFIKATOR)`, jeśli nazwa opisowa jest dostępna. W przeciwnym razie zwraca sam identyfikator.

4.5 Pobieranie i agregacja danych

Główną funkcją pobierającą pomiary jest `load_data()`. Przyjmuje nazwę bazy, schemat i tabelę, identyfikator zmiennej oraz początek i koniec zakresu jako timestampy Unix w sekundach.

Zapytanie SQL składa się z czterech etapów zapisanych jako CTE:

1. `filtered` – filtrowanie rekordów i obliczenie czasu w milisekundach,
2. `bounds` – wyznaczenie granic czasu pobranego zbioru,
3. `bucketed` – przypisanie rekordów do przedziałów,
4. `aggregated` – obliczenie wartości statystycznych w każdym przedziale.

4.5.1 Filtrowanie i obliczenie czasu

CTE `filtered` wybiera rekordy wskazanej zmiennej z zakresu:

```
VARIABLE = ?
TIMESTAMP_S >= ?
TIMESTAMP_S <= ?
```

Wartości są przekazywane jako parametry do `pandas.read_sql`. Ogranicza to konieczność ręcznego wstawiania wartości użytkownika do zapytania.

Zapytanie pobiera również kolumny `CALCULATION`, `STATUS`, `GUID` i `STRVALUE`. Czas w sekundach i milisekundach jest łączony w pomocniczą wartość:

```
TIMESTAMP_S * 1000 + COALESCE(TIMESTAMP_MS, 0)
```

Jeżeli wartość `TIMESTAMP_MS` jest pusta, przyjmowane jest zero. Filtrowanie zakresu w zapytaniu odbywa się na kolumnie `TIMESTAMP_S`, a więc z dokładnością do sekundy; milisekundy służą później m.in. do sortowania i wyznaczania czasu punktu.

4.5.2 Wyznaczanie granic i bucketów

CTE **bounds** wyznacza najwcześniejszy i najpóźniejszy czas w przefiltrowanym zbiorze. Następnie każdy rekord otrzymuje numer bucketa obliczany proporcjonalnie do położenia w zakresie:

$$bucket = \left\lfloor \frac{t - t_{\min}}{t_{\max} - t_{\min}} \cdot N \right\rfloor$$

gdzie t jest czasem rekordu w milisekundach, $t_{\min}$ i $t_{\max}$ oznaczają granice czasu, a N jest wartością `MAX_PLOT_POINTS`, obecnie równą 1000.

Numery bucketów zaczynają się od zera. Zgodnie z aktualnym wzorem najpóźniejszy rekord może otrzymać numer 1000, dlatego liczba niepustych bucketów może wynieść do 1001. Jeśli wszystkie rekordy mają ten sam czas, trafiają do bucketa 0. Warunek w zapytaniu chroni obliczenie przed dzieleniem przez zero.

4.5.3 Agregowanie wartości

CTE **aggregated** grupuje rekordy według zmiennej i numeru bucketa. Wiersze z pustą wartością **VALUE** są pomijane. Dla każdego bucketa obliczane są:

```
AVG(VALUE) AS VALUE_AVG
MIN(VALUE) AS VALUE_MIN
MAX(VALUE) AS VALUE_MAX
```

Kolumny **CALCULATION**, **STATUS**, **GUID** i **STRVALUE** są reprezentowane przez wartość **MIN** w grupie. Dla tych kolumn nie jest to statystyka pomiaru, lecz sposób wybrania jednej wartości reprezentującej bucket.

Zapytanie zapisuje również najwcześniejszy i najpóźniejszy czas w każdym buckecie. Czas punktu wynikowego jest wyznaczany jako środek pomiędzy tymi wartościami, a następnie rozdzielany ponownie na sekundy i milisekundy. Wynik jest sortowany chronologicznie według obu kolumn.

4.6 Konwersja wyniku w Pandas

Jeśli zapytanie nie zwróci rekordów, funkcja zwraca pusty **DataFrame** z oczekiwanymi kolumnami, w tym **time**, **VALUE_AVG**, **VALUE_MIN** i **VALUE_MAX**.

Dla niepustego wyniku funkcja:

- konwertuje kolumny czasu i wartości do typów numerycznych,
- tworzy kolumnę **time** z sekund i milisekund,
- przelicza czas zgodnie z **LOCAL_TZ**,
- usuwa wiersze bez poprawnego czasu lub wartości **AVG**,
- sortuje dane chronologicznie i resetuje indeks.

Wynikowa kolumna **time** jest pozbawiona informacji o strefie czasowej. W aktualnej konfiguracji **LOCAL_TZ** oznacza stałe przesunięcie UTC+1.

4.7 Cache zapytań

Funkcje pobierające bazy, tabele, zmienne, nazwy zmiennych i dane korzystają z `st.cache_data`. Wpisy zależą od argumentów funkcji, np. nazwy bazy, tabeli, zmiennej i zakresu czasu.

Cache list baz, tabel i zmiennych obowiązuje przez 600 sekund, a cache danych pomiarowych przez 300 sekund. Przycisk ręcznego odświeżania w `app.py` czyści cache funkcji `load_data()`.

5 Moduł state.py

Moduł **state.py** zarządza stanem sesji Streamlit, konfiguracją serii oraz zakresem czasu. Obsługuje również zaznaczanie zakresu na wykresie i cofanie kolejnych poziomów zoomu.

5.1 Inicjalizacja stanu sesji

Streamlit ponownie wykonuje skrypt po wielu działaniach użytkownika. Wartości, które powinny przetrwać takie ponowne uruchomienie, są przechowywane w `st.session_state`.

Funkcja `init_session_state()` ustawia wartość domyślną tylko wtedy, gdy dany klucz jeszcze nie istnieje. Przy pierwszym uruchomieniu tworzy jedną serię z identyfikatorem **UUID** i ustawia początkowy zakres na ostatnią godzinę.

Domyślnym trybem jest **Ostatnie dyskretne**, a domyślnym gotowym zakresem **1 godzina**. Stan sesji przechowuje również m.in. konfigurację serii, wartości widgetów, bieżący zakres, stan zoomu, historię zoomu i wersję klucza wykresu.

5.2 Tryby wyboru zakresu

Aplikacja obsługuje trzy tryby:

- **Ostatnie dyskretne** – wybór gotowego przedziału,
- **Ostatnie ciągłe** – podanie liczby sekund, minut, godzin i dni wstecz,
- **Własny zakres** – podanie daty i godziny początku oraz końca.

Gotowe zakresy zapisane w `QUICK_RANGES` to 10 minut, 1 godzina, 1 dzień, 1 tydzień, 1 miesiąc (30 dni) i 1 rok (365 dni). Bieżący zakres i zakres zapamiętany przed zoomem są przechowywane osobno:

```
time_start, time_end
time_start_original, time_end_original
```

5.3 Zapamiętywanie zakresu czasu

Funkcja `insert_time(start_time, end_time)` aktualizuje wartości widgetów zgodnie z wybranym trybem. W trybie dyskretnym wybiera gotowy zakres najbliższy przekazanej długości. W trybie ciągłym rozkłada długość zakresu na dni, godziny, minuty i sekundy. W trybie własnym aktualizuje pola daty i godziny.

Niezależnie od trybu funkcja zapisuje zakres jako aktualny oraz zapamiętany. Jest używana m.in. przy dodawaniu i usuwaniu serii, aby zachować ustawiony czas po rerunie Streamlit.

5.4 Dodawanie, usuwanie i resetowanie serii

Funkcja `add_series(max_series)` dodaje nową serię, jeśli liczba serii nie osiągnęła limitu. Każda nowa seria otrzymuje identyfikator UUID.

Funkcja `remove_series(series_id)` usuwa wybraną serię, jej konfigurację oraz zapisane klucze widgetów tabeli i zmiennej. Przed dodaniem lub usunięciem serii, jeśli zoom nie jest aktywny, zachowywany jest bieżący zakres czasu.

Funkcja `reset_series()` czyści konfiguracje wszystkich serii i tworzy jedną nową serię. Jest używana po zmianie bazy danych. Nie resetuje sama z siebie zakresu czasu.

5.5 Zaznaczenie zakresu i historia zoomu

Funkcja `handle_zoom_box_select(box_x)` obsługuje zaznaczenie prostokątne przekazane z wykresu. Oczekuje co najmniej dwóch wartości czasu. Jeśli zaznaczenie jest identyczne z ostatnio przetworzonym, funkcja je pomija, aby nie obsłużyć ponownie tego samego zdarzenia podczas kolejnego rerunu.

Końce zaznaczenia są porządkowane chronologicznie. Przy pierwszym zaznaczeniu zapamiętywany jest dotychczasowy tryb zakresu, a historia zoomu jest czyszczona. Przy kolejnym zaznaczeniu bieżący zakres jest dodawany do `zoom_history`, po czym ustawiany jest nowy zakres.

Funkcja oznacza zoom jako aktywny i zwiększa `chart_key_version`. Zwraca `True`, jeśli zakres został zmieniony, i `False`, jeśli zaznaczenie jest niepoprawne albo zostało już przetworzone.

Zaznaczenie zmienia zakres wykorzystywany do pobrania danych; nie jest wyłącznie wizualnym powiększeniem istniejących punktów.

5.6 Cofanie zoomu

Funkcja `undo_zoom()` cofa jeden poziom zoomu. Jeśli `zoom_history` zawiera wcześniejszy zakres, zostaje on zdjęty ze stosu i przywrócony.

Gdy historia jest pusta, funkcja przywraca tryb wyboru czasu i zakres sprzed pierwszego zoomu. Następnie wyłącza zoom, czyści informacje o ostatnim zaznaczeniu i zwiększa `chart_key_version`, aby odświeżyć stan komponentu wykresu.

Walidacja, czy początek wybranego zakresu nie jest późniejszy niż jego koniec, odbywa się w `app.py`.

6 Moduł charting.py

Moduł `charting.py` odpowiada za budowanie wykresu Plotly oraz renderowanie interfejsu związanego z wykresem. Obsługuje wiele serii i osi Y, ślady AVG oraz MIN/MAX, markery M1 i M2, podsumowanie danych, zdarzenia wykresu i okresowe odświeżanie.

Z modułem współpracują `state.py`, który obsługuje zmianę zakresu po zaznaczeniu, oraz `controls.html`, który dodaje dodatkowe sterowanie myszą i klawiszem spacji.

6.1 Klucze wykresu i stan widgetów

Funkcje `_widget_key()` i `_chart_keys()` tworzą klucze widgetów oraz wartości stanu dla opcji MIN/MAX, rozmieszczenia osi, markerów i komponentu wykresu. Domyślny identyfikator wykresu to `main_chart`; przy wielu wykresach każdy powinien mieć własny `chart_key`.

Klucz komponentu uwzględnia `chart_key_version`. Ta sama wartość jest ustawiana jako `uirevision` figury Plotly. Zmiana wersji pozwala odświeżyć stan wykresu po zmianie zakresu lub cofnięciu zoomu. Przy zwykłym rerunie stan interakcji Plotly może być zachowany.

6.2 Konwersja i formatowanie czasu

Funkcja `_to_naive_timestamp()` konwertuje wartość do `pandas.Timestamp` i usuwa informację o strefie czasowej. Niepoprawny lub pusty czas zamienia na `None`.

Funkcja `_to_python_datetime()` zwraca standardowy obiekt `datetime.datetime`, wykorzystywany m.in. do obsługi zaznaczeń. Funkcja `_format_dt()` formatuje czas jako `YYYY-MM-DD HH:MM:SS.mmm`.

6.3 Zakresy i rozmieszczenie osi Y

Funkcja `_get_y_range()` odczytuje zakres Y dla danej serii. Jeżeli aktywne jest skalowanie automatyczne albo zakres ręczny jest niepoprawny, zwraca `None`. W przeciwnym razie zwraca parę wartości granicznych, pod warunkiem że minimum jest mniejsze od maksimum.

Funkcja `_axis_for_series()` przypisuje serie do osi:

- pierwsza seria korzysta z osi po lewej stronie,
- druga seria domyślnie korzysta z osi po prawej,
- kolejne serie otrzymują dodatkowe osie po lewej stronie,
- opcja „Wszystkie osie po lewej” umieszcza wszystkie osie po lewej stronie.

Dodatkowe osie nakładają się na główną oś i są rozmieszczane z użyciem parametrów `AXIS_GAP` oraz `MAX_DOMAIN_START`. Tytuły i etykiety osi przyjmują kolor przypisany do serii.

6.4 Budowa figury Plotly

Funkcja `_build_figure()` tworzy figurę i dodaje do niej ślady danych. Dla każdej serii wymagane są kolumny:

```
time
VALUE_AVG
VALUE_MIN
VALUE_MAX
```

Puste serie lub serie bez wymaganych kolumn są pomijane. Dla poprawnej serii tworzona jest oś Y, główny ślad AVG oraz — jeśli użytkownik włączył odpowiednią opcję — ślady MIN i MAX.

Wykres ma wysokość 700 pikseli, korzysta z ciemnego szablonu Plotly, a jego oś X przedstawia czas. Zakres osi X pochodzi z argumentów `start_time` i `end_time`. Legenda jest ułożona poziomo pod wykresem.

Wykres korzysta m.in. z trybu podpowiedzi najbliższego punktu, obsługi kliknięć i zaznaczeń oraz początkowego trybu przeciągania `zoom`.

6.5 Ślady AVG, MIN i MAX

Funkcja `_add_series_trace()` dodaje ślad jako `go.Scattergl`. Konfiguruje jego dane, kolor, szerokość i styl linii, przypisaną oś Y, legendę, dane pomocnicze `customdata` oraz format podpowiedzi.

Ślad AVG jest przedstawiany jako linia z markerami i pojawia się w legendzie. Po włączeniu opcji „Pokaż zakres MIN/MAX” dodawane są półprzezroczyste, kropkowane linie MIN i MAX. Te ślady są ukryte w legendzie. Opcja MIN/MAX jest domyślnie włączona dla maksymalnie dwóch serii.

W aktualnym kodzie ślad MAX jest rysowany z kolumny `VALUE_MAX`, ale przekazuje do podpowiedzi etykietę MIN. Podpowiedź tego śladu może więc błędnie opisywać wartość jako MIN.

6.6 Podsumowanie serii

Pod wykresem wyświetlana jest tabela podsumowania zawierająca etykietę serii i tabeli, ostatnią wartość AVG, średnią z wartości AVG oraz minimum i maksimum wyznaczone z zagregowanych kolumn MIN i MAX.

Podsumowanie jest obliczane na podstawie danych przekazanych do wykresu, czyli danych już zagregowanych przez zapytanie SQL.

6.7 Odczyt zdarzeń wykresu

Funkcja `_get_attr()` odczytuje atrybut zarówno ze słownika, jak i z obiektu zwracanego przez Streamlit. Funkcje `_get_selection()`, `_get_selection_points()` i `_get_box_selections()` pobierają zaznaczone punkty oraz zaznaczenia prostokątne. Jeśli danych brakuje, zwracane są puste listy lub wartości domyślne.

Każdy punkt otrzymuje w `customdata` indeks swojej serii. Funkcja `_extract_series_idx()` odczytuje ten indeks. Jeśli go brakuje, funkcja `_series_index_from_curve()` próbuje ustalić serię na podstawie numeru śladu. W tym trybie zakładana jest jedna linia na serię bez MIN/MAX albo trzy ślady na serię z MIN/MAX.

6.8 Markery M1 i M2

Tryb markerów włącza opcja „Markery klikane”. Pierwszy kliknięty punkt ustawia M1, drugi M2. Każdy następny klik przesuwa dotychczasowe M2 na pozycję M1 i ustawia nowy punkt jako M2.

Zapisywane są czas, wartość, numer śladu i indeks serii. Podpis ostatniego kliknięcia zapobiega wielokrotnemu przetworzeniu tego samego zdarzenia podczas rerunów Streamlit.

M1 jest wyświetlany na zielono, a M2 na różowo. Dla każdego markera dodawana jest etykieta, pionowa linia wskazująca czas oraz pozioma linia wskazująca wartość. Markery są przypisywane do osi Y właściwej serii.

Tabela markerów pokazuje czas i wartość zaznaczonych punktów. Jeśli M1 i M2 należą do tej samej serii, podawane są także:

$$\Delta Y = Y_{M2} - Y_{M1}$$

$$\Delta t = t_{M2} - t_{M1}$$

$$\frac{\Delta Y}{\Delta t}$$

Czas Δt jest wyrażany w sekundach. Iloraz nie jest obliczany, jeśli oba markery mają ten sam czas. Gdy markery dotyczą różnych serii, ich wartości są prezentowane osobno, bez obliczania różnicy między nimi.

Przycisk „Wyczyść markery” usuwa M1 i M2. Markery są również czyszczone po wyłączeniu trybu markerów.

6.9 Zaznaczanie zakresu i zoom

Wykres jest renderowany przez `st.plotly_chart()` z obsługą zaznaczeń punktów i obszarów. Zaznaczenie prostokątne jest przekazywane do funkcji `handle_zoom_box_select()` z modułu `state.py`. Jeśli zakres został zaakceptowany, aplikacja wykonuje pełny rerun i pobiera dane ponownie dla nowego przedziału.

Zwykły zoom Plotly zmienia widok wykresu po stronie przeglądarki. Sam w sobie nie zmienia zakresu, dla którego aplikacja pobiera dane. Ponowne pobranie danych następuje po obsłużeniu zaznaczenia Box Select.

6.10 Sterowanie myszy i klawisza spacji w controls.html

Funkcja `install_controls()` wczytuje plik `controls.html` i osadza jego zawartość za pomocą `components.html()`. Plik jest otwierany ze ścieżki względnej względem bieżącego katalogu roboczego, dlatego powinien być dostępny podczas uruchamiania aplikacji.

Skrypt JavaScript dodaje obsługę zdarzeń do dokumentu nadrzędnego strony. Flaga w tym dokumencie zapobiega instalowaniu obsługi wielokrotnie. Nad wykresem blokowane jest też standardowe menu kontekstowe przeglądarki.

Sterowanie myszą jest następujące:

- lewy przycisk wykonuje zwykły zoom Plotly,
- prawy przycisk przełącza wykres na Box Select,
- środkowy przycisk przełącza wykres na Pan, czyli przesuwanie widoku.

Dla prawego i środkowego przycisku skrypt przełącza tryb Plotly, a następnie przekazuje zdarzenia myszy wymagane do zaznaczenia lub przesunięcia. Po zwolnieniu przycisku tryb wykresu wraca do zoomu.

Naciśnięcie spacji aktywuje przycisk automatycznego dopasowania widoku na pasku narzędzi Plotly. Obsługa myszy i klawisza spacji znajduje się w pliku HTML; przekazanie zaznaczonego zakresu do pobrania danych odbywa się w Pythonie.

6.11 Fragment Streamlit i odświeżanie

Funkcja `make_chart_fragment()` opakowuje renderowanie wykresu we fragment Streamlit. Interwał odświeżania jest przekazywany w sekundach; wartość `None` wyłącza odświeżanie okresowe.

Fragment odświeża część interfejsu zawierającą wykres, niezależnie od pozostałej części strony. Ręczne odświeżenie jest obsługiwane w `app.py`: czyści cache funkcji `load_data()` i ponownie uruchamia aplikację, aby pobrać dane z SQL Servera.